

Part 1 - API Gateway

Exercise 1

The main focus of the exercise was the Apache Kafka producer. That's why let me focus on that part only in the explanation.

Scala

Data generation for Apache Kafka consists of 2 blocks:

- records accumulation
 - This part is fully managed by Apache Kafka through these 2 configuration properties, `batch.size` and `linger.ms`. The former one defines the max number of bytes accumulated in the buffer. When this value is reached, the producer sends the batched records right away.
However, sometimes the producer can hardly fill the buffer in and it may be delivering a lot of small batches. If you want to reduce the risk, you can set a time value in the `linger.ms` parameter. That way, if the batch is not full, the producer will wait some extra time before sending it.

 Please notice that in the code I used small values for both parameters because for sake of simplicity, I'm creating a separate producer for each bunch of records. There won't be that much data coming and all records are known beforehand, so there is no reason to define an extra waiting time.

 Even though a single producer instance might be enough for most of the use cases, you can also try using multiple producer instances. You should only ensure that each HTTP request you serve gets the same producer instance. Otherwise you might end up with disordered records.
On the implementation side, you can manage these multiple instances with [Object Pool design pattern](#).
- flushing
 - It's responsible for pushing all batched records to the Apache Kafka broker.

Therefore, in the first step of our function the producer accumulates as many records it can. If the number grows too much, the producer can empty the accumulation buffer by sending all records accumulated so far. Otherwise, we need to explicitly do that by performing the flush action after the `foreach` loop:

```
def produceDataToKafka(visitsToIngest: VisitsToIngest) = {  
  visitsToIngest.visits.foreach(visit => {
```

```

    val eventJson = JsonSerializer.Instance.writeValueAsString(visit)
    KafkaProducerInstance.send(new ProducerRecord[String, String](
        OutputTopic, visit.visitId.toString, eventJson
    ),
    kafkaProducerCallbacksHolder.registerCallbackForMessage(eventJson))
  })

  KafkaProducerInstance.flush()
}

```

If we had a 100% guarantee that the records will always be delivered to the topic, we could stop here. However, it's not the case and we have to deal with the unexpected. That's why the `send()` method also has a callback. This callback interacts with all pending messages queue and removes all successfully delivered events:

```

class KafkaProducerCallbacksHolder {

  val eventsToDeliver = new mutable.HashSet[String]()

  class KafkaProducerCallback(event: String) extends Callback {

    // We keep the events management in a single place, only within this
    class
    eventsToDeliver.add(event)

    override def onCompletion(metadata: RecordMetadata, exception:
    Exception): Unit = {
      if (exception == null) {
        eventsToDeliver.remove(event)
      } else {
        exception.printStackTrace()
      }
    }
  }
}

```

In the end of the delivery, the `eventsToDeliver` contains only the unsuccessfully delivered events that, as the second part of the exercise states, we should write to a fallback storage. It's currently done by this function:

```

def saveUndeliveredEvents() = {
  if (kafkaProducerCallbacksHolder.eventsToDeliver.nonEmpty) {
    FileUtils.writeStringToFile(
    new File
    (s"/tmp/bde-3/homework/p1/e1/fallback_${System.currentTimeMillis()}.json
    "),
    kafkaProducerCallbacksHolder.eventsToDeliver.mkString("\n"), "UTF-8")
  }
}

```

Python

My implementation in Python is very similar. I also created a `KafkaDataProducer` class and referenced the callbacks holder:

```
class KafkaDataProducer:

    OUTPUT_TOPIC = 'raw_data'

    def __init__(self):
        self.producer = Producer({'bootstrap.servers': 'localhost:29092',
                                   'linger.ms': 10 * 1000, # 10 seconds
                                   'batch.num.messages': 10,
                                   'retries': 3
                                   })

        self.kafka_data_producer_callbacks_holder =
            KafkaDataProducerCallbacksHolder()
```

Now, whenever the API gets the visits to save, I'm calling this delivery method:

```
def produce_data_to_kafka(self, visits_to_ingest):
    for visit_json in visits_to_ingest:
        visit = json.loads(visit_json)
        self.producer.produce(
            topic=KafkaDataProducer.OUTPUT_TOPIC,
            value=bytes(visit_json, encoding='utf-8'),
            key=bytes(str(visit['visitId']), encoding='utf-8'),

            on_delivery=self.kafka_data_producer_callbacks_holder.register_event_to_
            deliver(visit_json)
        )

        self.producer.flush(timeout=20)
```

And I'm cleaning the successfully delivered events in a callback dedicated for each event:

```
class KafkaDataProducerCallbacksHolder:

    def __init__(self):
        self.events_to_deliver = []

    def register_event_to_deliver(self, event_payload):
        self.events_to_deliver.append(event_payload)
```

```
def handle_delivery_result(error, result: Message):
    if error:
        print('Failure' + error)
    else:
        print('success')
        self.events_to_deliver.remove(event_payload)

    return handle_delivery_result
```

It doesn't differ that much from my Scala version, does it? Even the fallback storage is managed the same way:

```
def save_undelivered_records(self):
    if self.kafka_data_producer_callbacks_holder.events_to_deliver:
        dir_name = '/tmp/bde-3/homework/p1/e1/'
        file_name = f'fallback_{round(time.time() * 1000)}.json'
        os.makedirs(dir_name, exist_ok=True)
        pathlib.Path(f'{dir_name}/{file_name}') \
            .write_text(
                '\n'.join([event for event in
                    self.kafka_data_producer_callbacks_holder.events_to_deliver]))
```

Part 2 - Batch layer

Exercise 1

The pipeline has 2 important places. The first of them is **data reading**. Even though it starts very easily with an Apache Kafka data source...

```
val dataFrame = sparkSession.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "localhost:29092")
    .option("subscribe", "visits")
    .option("startingOffsets", "EARLIEST")
    .load()
```

...it becomes more complex with each step. To start, the mapping part. Apache Kafka stores each value as an array of bytes and Apache Spark doesn't have the capacity to load it directly as a string. That's why the first transformation is a to-string cast.

Starting from that point we can extract the relevant information for the partitioning schema, so the eventTime column. Please notice that in my JSON schema I'm not using all the fields of the record because simply I don't need them all. It helps reduce some parsing and memory overhead since there is always a timestamp field extracted and allocated:

```
val VisitTimestampOnlySchema = StructType(Seq(
  StructField("eventTime", TimestampType)
))

val query = dataframe.selectExpr("CAST(value AS STRING)")
  .select($"value",
    functions.from_json($"value", VisitTimestampOnlySchema).as("data"))
```

Later, I'm using a foreachBatch sink to write the dataset to 2 various places. I could also use 2 streaming sinks and spark.streams.awaitAnyTermination() to wait for them to execute. However, it would involve having 2 fully executed streaming writes operation, from the input reading to the data writing, as in the following screenshot:

Completed Jobs (22)

Page: 1 Pages. Jump to . Show items in a page.

Job Id (Job Group) ▼	Description	Submitted	Stages:		Tasks (for all stages): Succeeded/Total
			Duration	Succeeded/Total	
21 (bed45ea8-e46d-4349-8800-b607a043a9cf)	id = a5be6bd8-3595-4934-b451-1f35cf71a9aa runId = bed45ea8-e46d-4349-8800-b607a043a9cf start at BatchSynchronizerStreamingQueries.scala:41	2023/03/24 05:26:02	0.2 s	1/1	1/1
20 (be3a42af-8ca1-4634-8252-3fe0535512e9)	id = f76216ed-348e-4f24-a03d-b86ca1a33ac0 runId = be3a42af-8ca1-4634-8252-3fe0535512e9 start at BatchSynchronizerStreamingQueries.scala:57	2023/03/24 05:26:01	0.8 s	1/1	1/1
19 (bed45ea8-e46d-4349-8800-b607a043a9cf)	Delta: id = a5be6bd8-3595-4934-b451-1f35cf71a9aa runId = bed45ea8-e46d-4349-8800-b607a043a9cf start at BatchSynchronizerStreamingQueries.scala:41	2023/03/24 05:25:59	3 s	3/3	53/53
18 (be3a42af-8ca1-4634-8252-3fe0535512e9)	Delta: id = f76216ed-348e-4f24-a03d-b86ca1a33ac0 runId = be3a42af-8ca1-4634-8252-3fe0535512e9 start at BatchSynchronizerStreamingQueries.scala:57	2023/03/24 05:25:58	2 s	3/3	53/53

When it comes to the foreachBatch, it's always a single Structured Streaming job per micro-batch, hence also a single checkpoint file that can simplify the day-to-day maintenance and guarantee each micro-batch always processes the same set of input (= data loaded only once).

Completed Jobs (113, only showing 112)

Page: 1 2 >

2 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id (Job Group) ▾	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
111 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	Delta: id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1f... start at BatchSynchronizer.scala:59	2023/03/24 05:32:50	1 s	3/3	57/57
110 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1fd04450... start at BatchSynchronizer.scala:59	2023/03/24 05:32:49	52 ms	1/1	1/1
109 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	Delta: id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1f... start at BatchSynchronizer.scala:59	2023/03/24 05:32:49	0.7 s	3/3	57/57
108 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1fd04450... start at BatchSynchronizer.scala:59	2023/03/24 05:32:48	0.6 s	1/1	1/1
107 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	Delta: id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1f... start at BatchSynchronizer.scala:59	2023/03/24 05:32:43	1.0 s	3/3	56/56
106 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1fd04450... start at BatchSynchronizer.scala:59	2023/03/24 05:32:43	59 ms	1/1	1/1
105 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	Delta: id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1f... start at BatchSynchronizer.scala:59	2023/03/24 05:32:42	0.9 s	3/3	56/56
104 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1fd04450... start at BatchSynchronizer.scala:59	2023/03/24 05:32:41	0.6 s	1/1	1/1
103 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	Delta: id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1f... start at BatchSynchronizer.scala:59	2023/03/24 05:32:40	1 s	3/3	55/55
102 (965d95c4-bcff-4ef9-ac1a-6cc1fd04450d)	id = 3c7fc8b5-eb35-4f0e-8b60-8537d0c185b8 runId = 965d95c4-bcff-4ef9-ac1a-6cc1fd04450... start at BatchSynchronizer.scala:59	2023/03/24 05:32:39	86 ms	1/1	1/1

Inside the foreachBatch, I'm first classifying the event as being valid or invalid with a simple if-else (CASE-WHEN in SQL) statement:

```
val datasetWithValidityFlag = dataset.withColumn("is_valid_event_time",  
functions.when($"data.eventTime".isNull, false).otherwise(true))
```

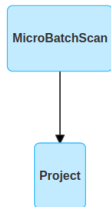
I'm not doing that before the foreachBatch to break the lineage up. Otherwise, the condition would rerun the from_json twice. You can see that in the execution plan below:

Details for Query 109

Submitted Time: 2023/03/24 05:39:18

Duration: 2 s

☐ Show the Stage ID and Task ID that corresponds to the max metric



Details

```
== Physical Plan ==  
Project (2)  
+- MicroBatchScan (1)  
  
(1) MicroBatchScan  
Output [7]: [key#7, value#8, topic#9, partition#10, offset#11L, timestamp#12, timestampType#13]  
class org.apache.spark.sql.kafka010.KafkaSourceProvider$KafkaScan  
  
(2) Project  
Output [3]: [cast(value#8 as string) AS value#21, from_json(StructField(event_time,TimestampType,true), cast(value#8 as string), Some(Europe/Paris)) AS data#23,  
isNotNull(from_json(StructField(event_time,TimestampType,true), cast(value#8 as string), Some(Europe/Paris)).event_time) AS is_valid_event_time#26]  
Input [7]: [key#7, value#8, topic#9, partition#10, offset#11L, timestamp#12, timestampType#13]
```

So, the next 2 parts for the writing are quite similar. In the first one I'm using the valid events to extract the partition column, like that:

```
datasetWithValidityFlag.filter("is_valid_event_time == true")
  .withColumn("year", functions.year($"data.eventTime"))
  .withColumn("month", functions.month($"data.eventTime"))
  .withColumn("day", functions.dayofmonth($"data.eventTime"))
  .withColumn("hour", functions.hour($"data.eventTime"))
  .write.format("delta").mode(SaveMode.Append)
  .partitionBy(
    "year", "month", "day", "hour"
  ).save(s"${dataOutputDir}/valid")
```

Whereas for the second one, I'm using the current execution time for that:

```
datasetWithValidityFlag.filter("is_valid_event_time == false")
  .withColumns(
    Map(
      "year" -> functions.year(functions.current_timestamp()),
      "month" -> functions.month(functions.current_timestamp()),
      "day" -> functions.dayofmonth(functions.current_timestamp()),
      "hour" -> functions.hour(functions.current_timestamp()),
    )
  ).write.format("delta").mode(SaveMode.Append)
  .partitionBy(
    "year", "month", "day", "hour").save(s"${dataOutputDir}/invalid")
```

Best,
Bartosz.